

TP PHP — POO & PDO

Gestion de Bibliothèque

Guide complet des fichiers avec explications detaillees

Objectif : Ce document explique chaque fichier PHP cree pour le TP. Pour chaque fichier, vous trouverez : le role du fichier, une explication des concepts utilises (POO, PDO, requetes preparees), et des notes pedagogiques pour comprendre POURQUOI on ecrit le code ainsi.

Structure du projet

Voici l'arborescence de tous les fichiers crees :

```
bibliotheque/  
connexion.php <- Connexion PDO (Singleton)  
index.php <- Page d'accueil avec statistiques  
auteurs.php <- Page CRUD : auteurs  
categories.php <- Page CRUD : categories  
livres.php <- Page CRUD : livres  
style.css <- Feuille de style HTML/CSS  
classes/  
Auteur.php <- Classe PHP pour les auteurs  
Categorie.php <- Classe PHP pour les categories  
Livre.php <- Classe PHP pour les livres
```

1. connexion.php — La connexion PDO

connexion.php

Role du fichier :

Ce fichier cree et gere la connexion a la base de donnees MySQL via PDO. Il utilise le patron de conception **Singleton** : on ne cree la connexion qu'une seule fois, et tous les autres fichiers reutilisent la meme instance.

Concepts cles expliques :

- **PDO (PHP Data Objects)** : c'est la couche d'abstraction PHP pour parler a n'importe quelle base de donnees (MySQL, PostgreSQL...) de facon securisee.
- **Singleton** : un patron de conception qui garantit qu'une classe n'a qu'une seule instance. Ici, on ne se connecte a MySQL qu'une seule fois.

- **private static \ \$instance** : variable de classe (pas d'objet) qui stocke l'unique instance.
- **private function __construct()** : le constructeur est prive, donc on ne peut pas faire 'new Connexion()' depuis l'exterieur.
- **PDO::ERRMODE_EXCEPTION** : en cas d'erreur SQL, PHP lance une exception au lieu d'echouer silencieusement.
- **PDO::FETCH_ASSOC** : les resultats des requetes sont retournees comme tableaux associatifs (ex: \ \$row['nom']).

```
class Connexion {
    private static \ $instance = null; // Une seule instance
    private \ $pdo; // L'objet PDO

    // Constructeur PRIVE : interdit new Connexion() depuis dehors
    private function __construct() {
        \ $dsn = 'mysql:host=localhost;dbname=bibliotheque;charset=utf8';
        \ $this->pdo = new PDO(\ $dsn, 'root', '');
    }

    // Methode pour obtenir l'unique instance
    public static function getInstance(): self {
        if (self::\ $instance === null) {
            self::\ $instance = new self(); // Cree UNE SEULE FOIS
        }
        return self::\ $instance;
    }

    public function getPDO(): PDO { return \ $this->pdo; }
}

// Usage dans les autres fichiers :
\ $pdo = Connexion::getInstance()->getPDO();
```

2. classes/Auteur.php — La classe Auteur

classes/Auteur.php

Role du fichier :

Cette classe regroupe toutes les operations CRUD (Create, Read, Update, Delete) pour la table **auteurs**. Chaque operation est une methode de la classe. C'est le principe d'encapsulation en POO : on cache les details SQL dans des methodes claires.

Les 4 operations CRUD expliquees :

Operation	Methode	SQL utilise	Retour
CREATE	ajouter()	INSERT INTO auteurs ...	bool
READ (liste)	listerTous()	SELECT * FROM auteurs	array

READ (un)	trouverParId()	SELECT ... WHERE id=:id	array/false
UPDATE	modifier()	UPDATE auteurs SET ...	bool
DELETE	supprimer()	DELETE FROM auteurs WHERE id=:id	bool

Requetes preparees — Pourquoi c'est OBLIGATOIRE :

Securite : Les requetes preparees protègent contre les injections SQL. Au lieu d'écrire directement la valeur dans le SQL, on utilise des marqueurs (:nom, :prenom) et PDO s'occupe d'échapper les caractères dangereux automatiquement.

```
// DANGEREUX (injection SQL possible) :
$sql = "SELECT * FROM auteurs WHERE nom = '" . $_POST['nom'] . "'";

// SECURISE avec requete preparee :
$stmt = $this->pdo->prepare(
    'INSERT INTO auteurs (nom, prenom, nationalite)'
    . ' VALUES (:nom, :prenom, :nationalite)'
);
$stmt->execute([
    ':nom' => $nom, // PDO echappe automatiquement
    ':prenom' => $prenom,
    ':nationalite' => $nationalite
]);
```

3. classes/Categorie.php — La classe Categorie

classes/Categorie.php

Role du fichier :

Identique en structure à la classe Auteur, mais pour la table **categories**. La table categories est plus simple (id + libelle), donc les methodes sont plus courtes. Ce fichier illustre bien le principe de réutilisation : même logique, données différentes.

Methodes de la classe :

- **ajouter(\$libelle)** : insere une nouvelle categorie avec son libelle.
- **listTous()** : retourne toutes les categories trieées par ordre alphabetique.
- **trouverParId(\$id)** : recupere une seule categorie par son id (utile pour pre-remplir le formulaire de modification).
- **modifier(\$id, \$libelle)** : met a jour le libelle d'une categorie existante.
- **supprimer(\$id)** : supprime une categorie de la base.

```
// Exemple : modifier une categorie
public function modifier(int $id, string $libelle): bool {
    $stmt = $this->pdo->prepare(
```

```
'UPDATE categories SET libelle=:libelle WHERE id=:id'

);

return \$stmt->execute([

'id' => \$id,

'libelle' => \$libelle

]);

}
```

4. classes/Livre.php — La classe Livre

classes/Livre.php

Role du fichier :

C'est la classe la plus importante du projet. Elle gere les livres qui ont des **cles etrangeres** vers les auteurs et les categories. La methode listerTous() utilise des jointures SQL pour recuperer directement le nom de l'auteur et le libelle de la categorie.

Concept cle : les jointures SQL (JOIN) :

LEFT JOIN permet de combiner plusieurs tables en une seule requete. Plutot que de faire 3 requetes separees (livres, puis auteurs, puis categories), on recupere tout en une seule requete SQL avec les JOIN.

```
// La methode listerTous() utilise LEFT JOIN pour tout recuperer d'un coup
public function listerTous(): array {
    \$sql = 'SELECT l.*,',
    ' CONCAT(a.prenom, " ", a.nom) AS auteur_nom,',
    ' c.libelle AS categorie_libelle',
    ' FROM livres l',
    ' LEFT JOIN auteurs a ON l.auteur_id = a.id',
    ' LEFT JOIN categories c ON l.categorie_id = c.id',
    ' ORDER BY l.titre';
    return \$this->pdo->query(\$sql)->fetchAll();
}

// Resultat : chaque ligne contient les colonnes de livres +
// 'auteur_nom' et 'categorie_libelle' directement utilisables.
```

5. auteurs.php — Page CRUD des auteurs

auteurs.php

Role du fichier :

C'est la page web visible par l'utilisateur. Elle combine du PHP (logique metier) et du HTML (affichage). Elle gere a la fois le formulaire d'ajout/modification et le tableau de liste. Un seul fichier fait tout : c'est le pattern **Page Controller**.

Logique PHP en haut de page — Comment ca marche :

```
// 1. On instancie la classe Auteur
\ $auteur = new Auteur();

// 2. Si le formulaire a ete envoye (methode POST)
if (\ $_SERVER['REQUEST_METHOD'] === 'POST') {
    \ $action = \ $_POST['action'] ?? '';

    if (\ $action === 'ajouter') {
        \ $auteur->ajouter(\ $_POST['nom'], \ $_POST['prenom'], \ $_POST['nationalite']);
    } elseif (\ $action === 'modifier') {
        \ $auteur->modifier(\ $_POST['id'], ...);
    } elseif (\ $action === 'supprimer') {
        \ $auteur->supprimer(\ $_POST['id']);
    }
}

// 3. Si un ?edit=5 est dans l'URL, on pre-remplit le formulaire
if (isset(\ $_GET['edit'])) {
    \ $editData = \ $auteur->trouverParId((int)\ $_GET['edit']);
}

// 4. On recupere la liste pour l'afficher
\ $liste = \ $auteur->listerTous();
```

Le HTML avec PHP inline — Affichage dynamique :

```
<!-- Le formulaire change selon si on ajoute ou modifie -->
<form method='POST'>

<!-- Champ cache pour dire quelle action effectuer -->
<input type='hidden' name='action'
value='<?= \ $editData ? 'modifier' : 'ajouter' ?>'>

<!-- Si modification, on envoie aussi l'ID -->
<?php if (\ $editData): ?>
<input type='hidden' name='id' value='<?= \ $editData['id'] ?>'>
<?php endif; ?>

<input type='text' name='nom'
value='<?= htmlspecialchars(\ $editData['nom'] ?? '') ?>'>
</form>

<!-- htmlspecialchars() : protege contre les failles XSS -->
```

6. livres.php — Page CRUD des livres

livres.php

Role du fichier :

La page la plus complexe. Le formulaire de livre necessite des menus deroulants (SELECT) pour choisir l'auteur et la categorie. On charge donc les trois classes en meme temps pour avoir acces aux donnees de chaque table.

Chargement de plusieurs classes :

```
// On a besoin des 3 classes en meme temps
require_once 'classes/Livre.php';
require_once 'classes/Auteur.php';
require_once 'classes/Categorie.php';

\$_livre = new Livre();
\$_auteur = new Auteur();
\$_categorie = new Categorie();

// Pour les menus deroulants du formulaire
\$_auteurs = \$_auteur->listerTous(); // liste pour <select>
\$_categories = \$_categorie->listerTous(); // liste pour <select>
```

Menu deroulant dynamique avec pre-selection (modification) :

```
<select name='auteur_id' required>
<?php foreach (\$_auteurs as \$_a): ?>
<option value='<?= \$_a['id'] ?>'
<?= (\$_editData['auteur_id'] == \$_a['id']) ? 'selected' : '' ?>>
<?= htmlspecialchars(\$_a['prenom'] . ' ' . \$_a['nom']) ?>
</option>
<?php endforeach; ?>
</select>

<!-- 'selected' pre-selectionne l'auteur actuel lors d'une modification -->
```

7. style.css — La feuille de style

style.css

Role du fichier :

Le fichier CSS donne l'apparence visuelle a toutes les pages. Toutes les pages HTML incluent ce fichier via la balise <link>. Voici les elements CSS importants :

Selecteur CSS	Ce qu'il style
nav	La barre de navigation en haut
.container	Le bloc central de contenu (largeur max)
.card	Les blocs blancs avec ombre (formulaires et tableaux)
table	Les tableaux de donnees
.btn-edit	Bouton de modification (orange)

.btn-delete	Bouton de suppression (rouge)
.alert	Message de confirmation vert
.stat-card	Cartes statistiques de la page d'accueil

8. Recapitulatif — Concepts POO utilises

Concept POO	Ou on l'utilise	Pourquoi
Classe (class)	Auteur, Categorie, Livre, Connexion	Regrouper donnees + comportements
Encapsulation	Attributs private dans Connexion	Proteger les donnees internes
Methodes publiques	ajouter(), modifier(), etc.	Interface pour interagir avec l'objet
Constructeur (__construct)	Toutes les classes	Initialisation automatique a la creation
Singleton (static)	Connexion::getInstance()	Une seule connexion PDO partagee
Type hints (: PDO)	getPDO(): PDO	Typage strict pour eviter les erreurs
require_once	En haut de chaque fichier	Inclure une classe sans la dupliquer

9. Comment faire fonctionner le projet

Etape 1 — Base de donnees	Ouvrir phpMyAdmin, executer le SQL de creation (CREATE DATABASE bibliotheque; etc.). Creer les tables auteurs, categories, livres.
Etape 2 — Configurer connexion.php	Modifier les valeurs de \$username et \$password dans connexion.php selon votre configuration WAMP/XAMPP. Par default : username='root', password=''.
Etape 3 — Copier les fichiers	Copier tout le dossier bibliotheque/ dans le dossier www/ (WAMP) ou htdocs/ (XAMPP).
Etape 4 — Ouvrir dans le navigateur	Aller sur http://localhost/bibliotheque/index.php et l'application doit fonctionner.
Etape 5 — Tester le CRUD	Ajouter d'abord des auteurs, puis des categories, puis des livres (car les livres ont besoin des deux).

Conseil : Si vous avez une erreur de connexion, verifiez que votre serveur MySQL est bien demarre (WAMP/XAMPP en vert), et que le nom de la base de donnees est bien 'bibliotheque' (sans accent).